

THE AGENT WHO SHIFTED WITH THE CONTEXT

✓ Learning objectives

Upon completing this lab, you will be able to implement a developmental agent driven by interactional motivation that adapts its next action based on the context of the previously enacted interaction.

✓ Define the Interaction class

Let's define an Interaction class that will be useful to initialize the agent and to memorize the context

```
class Interaction:
    """An interaction is a tuple (action, outcome) with a valence"""
    def __init__(self, action, outcome, valence):
        self.action = action
        self.outcome = outcome
        self.valence = valence

    def key(self):
        """ The key to find this interaction in the dictionary is the string '<action><outcome>'. """
        return f"{self.action}{self.outcome}"

    def __str__(self):
        """ Print interaction in the form '<action><outcome:<valence>' for debug. """
        return f"{self.action}{self.outcome}:{self.valence}"

    def __eq__(self, other):
        """ Interactions are equal if they have the same key """
        return self.key() == other.key()
```

✓ Define the Agent class

The agent is initialized with the list of interactions

The previous action and the predicted outcome are memorized in the attribute `_intended_interaction`.

```
class Agent:
    """Creating our agent"""
    def __init__(self, _interactions):
        """ Initialize the dictionary of interactions"""
        self._interactions = {interaction.key(): interaction for interaction in _interactions}
        self._intended_interaction = self._interactions["00"]

    def action(self, _outcome):
        """ Tracing the previous cycle """
        previous_interaction = self._interactions[f"{self._intended_interaction.action}{_outcome}"]
        print(f"Action: {self._intended_interaction.action}, Prediction: {self._intended_interaction.outcome}, Outcome: {_outcome}, "
              f"Prediction: {self._intended_interaction.outcome == _outcome}, Valence: {previous_interaction.valence}")

        """ Computing the next interaction to try to enact """
        # TODO: Implement the agent's decision mechanism
        intended_action = 0
        # TODO: Implement the agent's prediction mechanism
        intended_outcome = 0
        # Memorize the intended interaction
        self._intended_interaction = self._interactions[f"{intended_action}{intended_outcome}"]
        return intended_action
```

✓ Environment1 class

```
class Environment1:
    """ In Environment 1, action 0 yields outcome 0, action 1 yields outcome 1 """
    def outcome(self, _action):
        # return int(input("entre 0 1 ou 2"))
        if _action == 0:
            return 0
        else:
            return 1
```

✓ Environment2 class

```
class Environment2:
```

```

""" In Environment 2, action 0 yields outcome 1, action 1 yields outcome 0 """
def outcome(self, _action):
    if _action == 0:
        return 1
    else:
        return 0

```

Environment3 class

Environment 3 yields outcome 1 only when the agent alternates actions 0 and 1

```

class Environment3:
    """ Environment 3 yields outcome 1 only when the agent alternates actions 0 and 1 """
    def __init__(self):
        """ Initializing Environment3 """
        self.previous_action = 0

    def outcome(self, _action):
        if _action == self.previous_action:
            _outcome = 0
        else:
            _outcome = 1
        self.previous_action = _action
        return _outcome

```

Initialize the interactions

```

interactions = [
    Interaction(0,0,-1),
    Interaction(0,1,1),
    Interaction(1,0,-1),
    Interaction(1,1,1),
    Interaction(2,0,-1),
    Interaction(2,1,1)
]

```

Interactions are initialized with their action, their outcome, and their valence:

	outcome 0	outcome 1
action 0	-1	1
action 1	-1	1
action 2	-1	1

✓ Instantiate the agent

```
a = Agent(interactions)
```

✓ Instantiate the environment

```
e = Environment3()
```

✓ Test run the simulation

```
outcome = 0
for i in range(10):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1)
```

Observe that in Environment3, the agent obtains only negative valences. To obtain a positive valence, it must select a different action on each interaction cycle.

✓ PRELIMINARY EXERCISE

Execute the agent in Environment1. Observe that it obtains a negative valence.

Execute the agent in Environment2. Observe that it obtains a positive valence.

Now you see the goal of this assignment: design an agent that can obtain positive valences when it is run either in Environment1 or in Environment2 or in Environment3.

✓ ASSIGNMENT

Implement Agent4 that obtains positive valences in either Environment 1, 2, or 3.

Agent4 must be able to predict the outcome resulting from its next action depending on the context of the previous interaction. Based on this prediction, it must select the action that will yield the highest valence.

To do so, at the end of cycle t , Agent4 must memorize $\text{interaction}_t = (\text{action}_t, \text{outcome}_t)$ that was just enacted. The agent must choose the next interaction_{t+1} based on interaction_t (the context). For each possible action_{t+1} , the agent must predict the expected outcome_{t+1} . Based on this prediction, it must select the action that yields the highest valence_{t+1} .

✓ Create Agent4 by overriding the class Agent

You may add any attribute and method you deem usefull to the class Agent4

```
class Agent4(Agent):

    def __init__(self, _interactions):
        super().__init__(_interactions)

        # Mémoire des paires :
        # [interaction(t-1), interaction(t)]
        self.memoire = []

        # Dernière interaction réalisée
        self.interaction_precedente = None

    def predire_outcome(self, contexte, action):

        # Chercher ce qui s'est déjà passé
        # dans le même contexte avec la même action
```

```

for interaction_avant, interaction_apres in reversed(self.memoire):

    if (interaction_avant == contexte
        and interaction_apres.action == action):

        return interaction_apres.outcome

# Action jamais testée dans ce contexte
return None

def action(self, _outcome):

    # Interaction réellement réalisée
    interaction_actuelle = self._interactions[
        f"{self._intended_interaction.action}_{_outcome}"
    ]

    # Affichage du cycle
    print(
        f"Action: {self._intended_interaction.action}, "
        f"Prediction: {self._intended_interaction.outcome}, "
        f"Outcome: {_outcome}, "
        f"Prediction: {self._intended_interaction.outcome == _outcome}, "
        f"Valence: {interaction_actuelle.valence}"
    )

    # Mémoriser la paire :
    # [interaction précédente, interaction actuelle]
    if self.interaction_precedente is not None:
        self.memoire.append(
            [self.interaction_precedente, interaction_actuelle]
        )

    # L'interaction actuelle devient le contexte
    self.interaction_precedente = interaction_actuelle
    contexte = interaction_actuelle

    # Variables pour chercher la meilleure action
    meilleure_action = None
    meilleur_outcome = None
    meilleure_valence = float("-inf")

    actions_inconnues = []

    # Tester mentalement les trois actions possibles

```

```

for action in [0, 1, 2]:

    outcome_predit = self.predire_outcome(contexte, action)

    # Cette action n'a encore jamais été essayée
    # dans ce contexte
    if outcome_predit is None:
        actions_inconnues.append(action)

    else:
        interaction_predite = self._interactions[
            f"{action}{outcome_predit}"
        ]

        valence = interaction_predite.valence

        if valence > meilleure_valence:
            meilleure_valence = valence
            meilleure_action = action
            meilleur_outcome = outcome_predit

# Explorer d'abord une action inconnue
if len(actions_inconnues) > 0:
    intended_action = actions_inconnues[0]

    # On ne connaît pas encore son outcome,
    # donc prédiction 0 par défaut
    intended_outcome = 0

# Si toutes les actions sont connues,
# choisir celle avec la meilleure valence
else:
    intended_action = meilleure_action
    intended_outcome = meilleur_outcome

# Interaction prévue pour le prochain cycle
self._intended_interaction = self._interactions[
    f"{intended_action}{intended_outcome}"
]

return intended_action

```

✓ Test your Agent4 in Environment1

```

a = Agent4(interactions)
e = Environment1()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)

```

```

Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1

```

✓ Test your Agent4 in Environment2

```

a = Agent4(interactions)
e = Environment2()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)

```

```

Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1

```

```
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
```

✓ Test your Agent4 in Environment3

```
a = Agent4(interactions)
e = Environment3()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction: True, Valence: 1
```

✓ Test your Agent4 with interactions that have other valences

Replace the valences of interactions with your choice in the code below

```
# Choose different valence of interactions
interactions = [
    Interaction(0,0,1),
    Interaction(0,1,0),
    Interaction(1,0,-1),
    Interaction(1,1,1),
    Interaction(2,0,-1),
    Interaction(2,1,1)
]
# Run the agent
a = Agent4(interactions)
e = Environment3()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 0
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 0
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 0
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 1, Prediction: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: 0
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
```

Test your agent in the Turtle environment

✓ Valences Turtle

Pour l'environnement Turtle, je choisis une valence positive lorsque la tortue ne touche pas le bord (outcome 0) et une valence négative lorsqu'elle entre en collision avec le bord (outcome 1).

```
# Valences pour l'environnement Turtle :
# outcome 0 = pas de collision -> positif
# outcome 1 = collision -> négatif

interactions = [
    Interaction(0,0, 1),
    Interaction(0,1,-1),
    Interaction(1,0, 1),
    Interaction(1,1,-1),
    Interaction(2,0, 1),
    Interaction(2,1,-1)
]
```

✓ Install the turtle environment

```
# @title Install the turtle environment
!pip3 install ColabTurtle
from ColabTurtle.Turtle import *
```

```
Collecting ColabTurtle
  Downloading ColabTurtle-2.1.0.tar.gz (6.8 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: ColabTurtle
  Building wheel for ColabTurtle (setup.py) ... done
  Created wheel for ColabTurtle: filename=colabturtle-2.1.0-py3-none-any.whl size=7674 sha256=193b69b53ff7862fc87abc8bd8be1aba8af1ad27feb440321cc0
  Stored in directory: /root/.cache/pip/wheels/8d/30/a2/2b8e815bd3673383f33c0dd80d33ef7651a7c257c528515158
Successfully built ColabTurtle
Installing collected packages: ColabTurtle
Successfully installed ColabTurtle-2.1.0
```

✓ Initialize the turtle environment

```
# @title Initialize the turtle environment

BORDER_WIDTH = 20

class ColabTurtleEnvironment:

    def __init__(self):
```

```
""" Creating the Turtle window """
bgcolor("lightGray")
penup()
goto(window_width() / 2, window_height()/2)
face(0)
pendown()
color("green")
```

```
def outcome(self, action):
    """ Enacting an action and returning the outcome """
    _outcome = 0
    for i in range(10):
        # _outcome = 0
        if action == 0:
            # move forward
            forward(10)
        elif action == 1:
            # rotate left
            left(4)
            forward(2)
        elif action == 2:
            # rotate right
            right(4)
            forward(2)

    # Bump on screen edge and return outcome 1
    if xcor() < BORDER_WIDTH:
        goto(BORDER_WIDTH, ycor())
        _outcome = 1
    if xcor() > window_width() - BORDER_WIDTH:
        goto(window_width() - BORDER_WIDTH, ycor())
        _outcome = 1
    if ycor() < BORDER_WIDTH:
        goto(xcor(), BORDER_WIDTH)
        _outcome = 1
    if ycor() > window_height() - BORDER_WIDTH:
        goto(xcor(), window_height() - BORDER_WIDTH)
        _outcome = 1

    # Change color
    if _outcome == 0:
        color("green")
    else:
        # Finit l'interaction
        color("red")
        # if action == 0:
```

```

        #         break
    if action == 1:
        for j in range(10):
            left(4)
    elif action == 2:
        for j in range(10):
            right(4)
    break

return _outcome

```

✓ Run the turtle environment

```

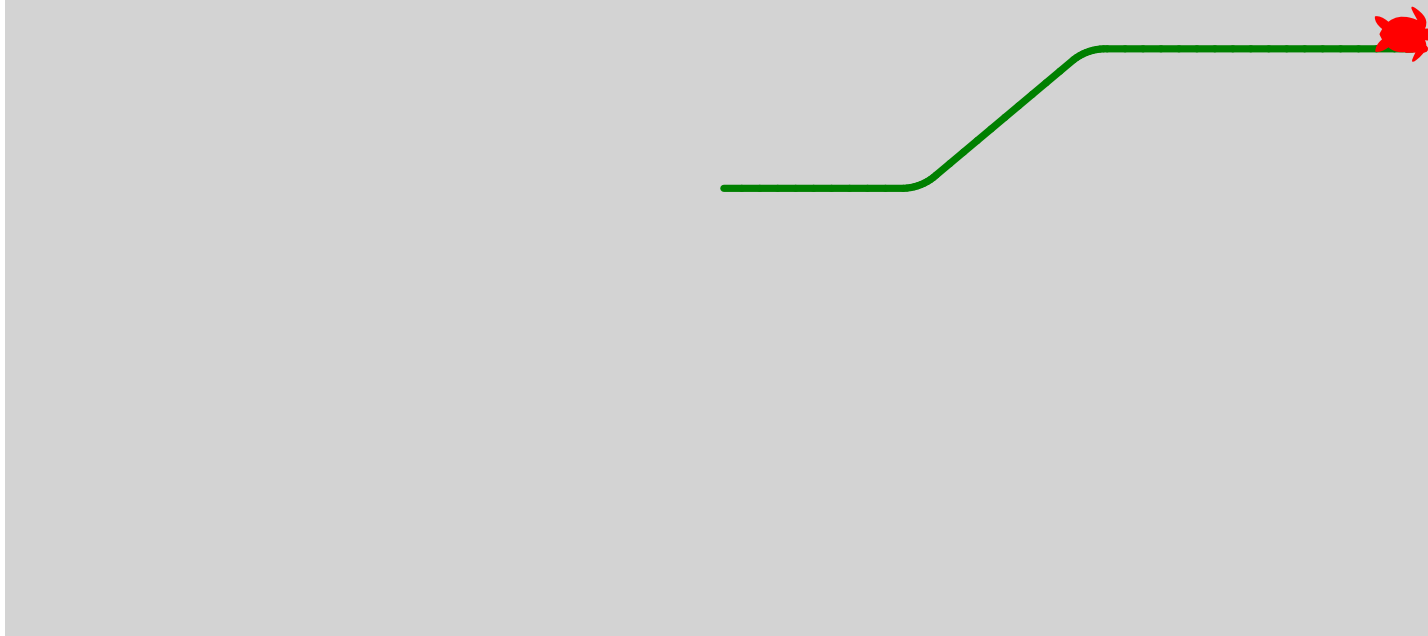
# @title Run the turtle environment
initializeTurtle()

# Parameterize the rendering
bgcolor("lightGray")
penup()
goto(window_width() / 2, window_height()/2)
face(0)
pendown()
color("green")
speed(10)

a = Agent4(interactions)
e = ColabTurtleEnvironment()

outcome = 0
for i in range(10):
    action = a.action(outcome)
    outcome = e.outcome(action)

```



```
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 1, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
```

✓ Report

Explain what you programmed and what results you observed. Export this document as PDF including your code, the traces you obtained, and your explanations below (no more than a few paragraphs):

Implémentation

J'ai implémenté Agent4 en ajoutant une mémoire des paires d'interactions successives. L'agent mémorise une paire [interaction précédente, interaction actuelle], où une interaction contient l'action, l'outcome et sa valence. Cette mémoire lui permet de prédire l'outcome d'une action en fonction du contexte, c'est-à-dire de l'interaction précédente. Pour chaque action possible, il recherche l'outcome déjà observé dans ce contexte, calcule la valence de l'interaction prédite et choisit l'action ayant la meilleure valence. Lorsqu'une action n'a pas encore été essayée dans un contexte, l'agent l'explore afin d'apprendre son résultat.

Résultats observés

Dans Environment1 et Environment2, après une phase d'exploration, Agent4 apprend les conséquences de ses actions et finit par choisir une action produisant une valence positive. Dans Environment3, le résultat dépend de l'action précédente. L'agent apprend progressivement ce contexte et, avec les valences initiales, il finit par alterner entre les actions 0 et 1 afin d'obtenir régulièrement l'outcome 1 et une valence positive.

Avec les nouvelles valences choisies, l'agent adapte son comportement et privilégie les interactions de valence positive. Dans l'environnement Turtle, j'ai choisi une valence positive lorsque la tortue ne touche pas le bord (outcome 0) et négative lorsqu'elle se cogne (outcome 1). La trace montre que l'agent obtient d'abord des interactions positives, puis rencontre le bord. Ces collisions produisent alors une valence négative et sont mémorisées par l'agent pour être prises en compte lorsqu'il rencontre à nouveau les mêmes contextes.